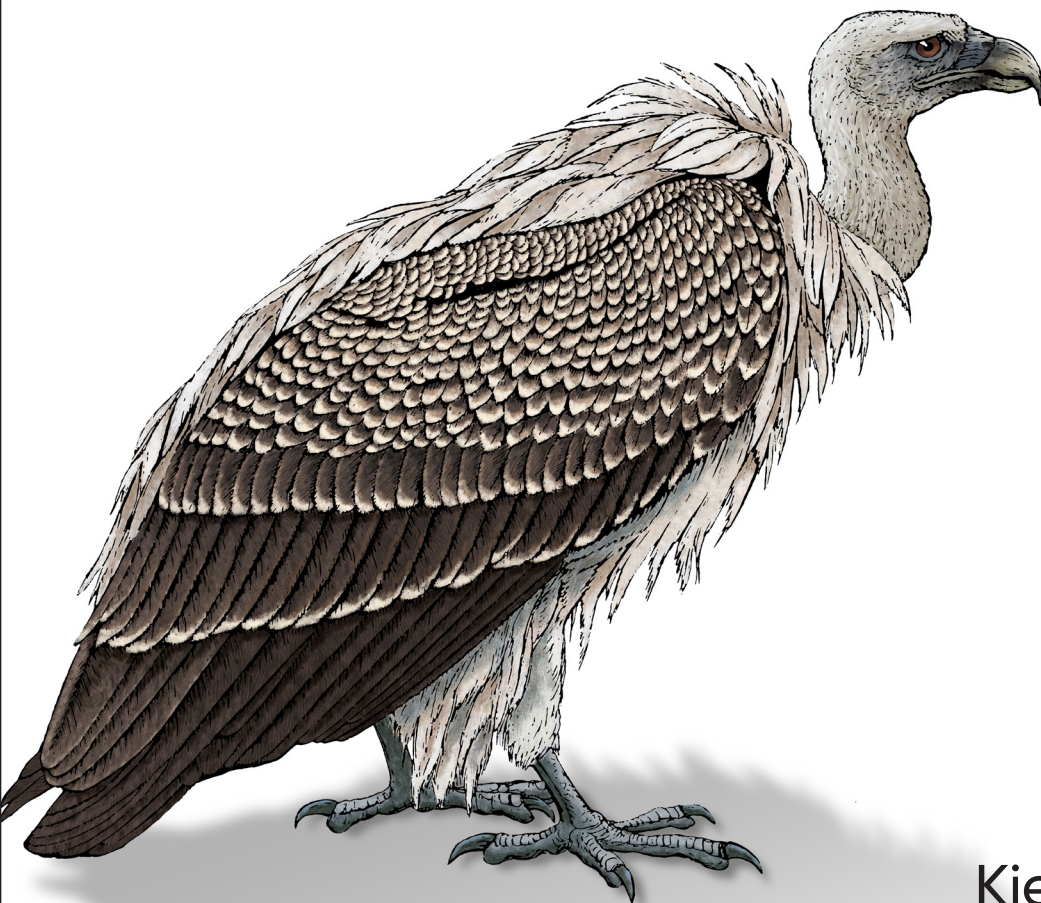


O'REILLY®

Second
Edition

Infrastructure as Code

Dynamic Systems for the Cloud Age



Kief Morris

Infrastructure as Code

Six years ago, Infrastructure as Code was a new concept. Today, as even banks and other conservative organizations plan moves to the cloud, development teams for companies worldwide are attempting to build large infrastructure codebases. With this practical book, Kief Morris of ThoughtWorks shows you how to effectively use principles, practices, and patterns pioneered by DevOps teams to manage cloud-age infrastructure.

Ideal for system administrators, infrastructure engineers, software developers, team leads, and architects, this updated edition demonstrates how you can exploit cloud and automation technology to make changes easily, safely, quickly, and responsibly. You'll learn how to define everything as code and apply software design and engineering practices to build your system from small, loosely coupled pieces.

This book covers:

- **Foundations:** Use Infrastructure as Code to drive continuous change and raise the bar of operational quality, using tools and technologies to build cloud-based platforms
- **Working with infrastructure stacks:** Learn how to define, provision, test, and continuously deliver changes to infrastructure resources
- **Working with servers and other platforms:** Use patterns to design provisioning and configuration of servers and clusters
- **Working with large systems and teams:** Learn workflows, governance, and architectural patterns to create and manage infrastructure elements

"Infrastructure as Code practices have evolved from managing servers to managing complete stacks, but this new power comes at the cost of complexity. This book will take you beyond the commands to the design patterns behind good practices and the how-tos of next-level automation."

—Patrick Debois
DevOpsDays Founder

Kief Morris is global director of cloud engineering at ThoughtWorks. He helps organizations and teams explore better ways to apply cloud and infrastructure technology to deliver stronger value more quickly and reliably. He's been designing, building, and running automated IT server infrastructure for over 20 years, starting out with shell scripts and Perl and moving on to CFengine, Puppet, Chef, and Terraform among other technologies as they've emerged.

SYSTEM ADMINISTRATION

US \$69.99

CAN \$92.99

ISBN: 978-1-098-11467-1



Praise for *Infrastructure as Code*

Infrastructure as Code practices have evolved from managing servers to managing complete stacks, but this new power comes at the cost of complexity. This book will take you beyond the commands to the design patterns behind good practices and the how-to's of next-level automation.

—Patrick Debois, *DevOpsDays* Founder

Infrastructure as Code sits at the intersection of multiple software engineering domains. This book regroups and distills the relevant best practices from each domain to give readers an accelerated course on infrastructure automation.

—Carlos Condé, *VP of Engineering at Sweetgreen*

A down-to-earth guide for navigating the landscape of cloud infrastructure with principles, practices and patterns.

—Effy Elden, *Technologist at ThoughtWorks*

SECOND EDITION

Infrastructure as Code

Dynamic Systems for the Cloud Age

Kief Morris

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Infrastructure as Code

by Kief Morris

Copyright © 2021 Kief Morris. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: John Devins

Development Editor: Virginia Wilson

Production Editor: Kate Galloway

Copyeditor: Kim Cofer

Proofreader: nSight, Inc.

Indexer: Potomac Indexing, LLC

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: John Francis Amalanathan

June 2016: First Edition

December 2020: Second Edition

Revision History for the Second Edition

2020-11-17: First Release

2021-01-15: Second Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098114671> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Infrastructure as Code*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

This work is part of a collaboration between O'Reilly and Linode. See our [statement of editorial independence](#).

978-1-098-11467-1

[GP]

Table of Contents

Preface.....	xv
--------------	----

Part I. Foundations

1. What Is Infrastructure as Code?.....	1
From the Iron Age to the Cloud Age	2
Infrastructure as Code	4
Benefits of Infrastructure as Code	4
Use Infrastructure as Code to Optimize for Change	4
Objection: “We don’t make changes often enough to justify automating them”	5
Objection: “We should build first and automate later”	6
Objection: “We must choose between speed and quality”	7
The Four Key Metrics	9
Three Core Practices for Infrastructure as Code	9
Core Practice: Define Everything as Code	10
Core Practice: Continuously Test and Deliver All Work in Progress	10
Core Practice: Build Small, Simple Pieces That You Can Change Independently	11
Conclusion	11
2. Principles of Cloud Age Infrastructure.....	13
Principle: Assume Systems Are Unreliable	14
Principle: Make Everything Reproducible	14
Pitfall: Snowflake Systems	15
Principle: Create Disposable Things	16
Principle: Minimize Variation	17

Configuration Drift	17
Principle: Ensure That You Can Repeat Any Process	20
Conclusion	21
3. Infrastructure Platforms.....	23
The Parts of an Infrastructure System	23
Infrastructure Platforms	25
Infrastructure Resources	27
Compute Resources	28
Storage Resources	29
Network Resources	31
Conclusion	33
4. Core Practice: Define Everything as Code.....	35
Why You Should Define Your Infrastructure as Code	35
What You Can Define as Code	36
Choose Tools with Externalized Configuration	36
Manage Your Code in a Version Control System	37
Infrastructure Coding Languages	38
Infrastructure Scripting	39
Declarative Infrastructure Languages	41
Programmable, Imperative Infrastructure Languages	43
Declarative Versus Imperative Languages for Infrastructure	44
Domain-Specific Infrastructure Languages	44
General-Purpose Languages Versus DSLs for Infrastructure	46
Implementation Principles for Defining Infrastructure as Code	46
Separate Declarative and Imperative Code	46
Treat Infrastructure Code Like Real Code	47
Conclusion	48

Part II. Working with Infrastructure Stacks

5. Building Infrastructure Stacks as Code.....	51
What Is an Infrastructure Stack?	51
Stack Code	53
Stack Instance	53
Configuring Servers in a Stack	54
Low-Level Infrastructure Languages	54
High-Level Infrastructure Languages	55
Patterns and Antipatterns for Structuring Stacks	56
Antipattern: Monolithic Stack	56

Pattern: Application Group Stack	59
Pattern: Service Stack	60
Pattern: Micro Stack	62
Conclusion	63
6. Building Environments with Stacks.	65
What Environments Are All About	65
Delivery Environments	66
Multiple Production Environments	66
Environments, Consistency, and Configuration	67
Patterns for Building Environments	68
Antipattern: Multiple-Environment Stack	68
Antipattern: Copy-Paste Environments	70
Pattern: Reusable Stack	72
Building Environments with Multiple Stacks	74
Conclusion	75
7. Configuring Stack Instances.	77
Using Stack Parameters to Create Unique Identifiers	79
Example Stack Parameters	79
Patterns for Configuring Stacks	80
Antipattern: Manual Stack Parameters	81
Pattern: Stack Environment Variables	82
Pattern: Scripted Parameters	84
Pattern: Stack Configuration Files	87
Pattern: Wrapper Stack	90
Pattern: Pipeline Stack Parameters	93
Pattern: Stack Parameter Registry	96
Configuration Registry	99
Implementing a Configuration Registry	100
Single or Multiple Configuration Registries	102
Handling Secrets as Parameters	102
Encrypting Secrets	102
Secretless Authorization	103
Injecting Secrets at Runtime	103
Disposable Secrets	104
Conclusion	104
8. Core Practice: Continuously Test and Deliver.	105
Why Continuously Test Infrastructure Code?	106
What Continuous Testing Means	106
What Should We Test with Infrastructure?	108

Challenges with Testing Infrastructure Code	110
Challenge: Tests for Declarative Code Often Have Low Value	110
Challenge: Testing Infrastructure Code Is Slow	113
Challenge: Dependencies Complicate Testing Infrastructure	114
Progressive Testing	115
Test Pyramid	116
Swiss Cheese Testing Model	118
Infrastructure Delivery Pipelines	119
Pipeline Stages	120
Scope of Components Tested in a Stage	121
Scope of Dependencies Used for a Stage	121
Platform Elements Needed for a Stage	122
Delivery Pipeline Software and Services	123
Testing in Production	125
What You Can't Replicate Outside Production	125
Managing the Risks of Testing in Production	126
Conclusion	127
9. Testing Infrastructure Stacks.....	129
Example Infrastructure	129
The Example Stack	130
Pipeline for the Example Stack	131
Offline Testing Stages for Stacks	131
Syntax Checking	132
Offline Static Code Analysis	132
Static Code Analysis with API	133
Testing with a Mock API	133
Online Testing Stages for Stacks	134
Preview: Seeing What Changes Will Be Made	134
Verification: Making Assertions About Infrastructure Resources	135
Outcomes: Proving Infrastructure Works Correctly	137
Using Test Fixtures to Handle Dependencies	137
Test Doubles for Upstream Dependencies	139
Test Fixtures for Downstream Dependencies	139
Refactor Components So They Can Be Isolated	141
Life Cycle Patterns for Test Instances of Stacks	142
Pattern: Persistent Test Stack	142
Pattern: Ephemeral Test Stack	143
Antipattern: Dual Persistent and Ephemeral Stack Stages	145
Pattern: Periodic Stack Rebuild	146
Pattern: Continuous Stack Reset	147
Test Orchestration	149

Support Local Testing	149
Avoid Tight Coupling with Pipeline Tools	150
Test Orchestration Tools	150
Conclusion	151

Part III. Working with Servers and Other Application Runtime Platforms

10. Application Runtimes.....	155
Cloud Native and Application-Driven Infrastructure	156
Application Runtime Targets	157
Deployable Parts of an Application	157
Deployment Packages	158
Deploying Applications to Servers	159
Packaging Applications in Containers	159
Deploying Applications to Server Clusters	160
Deploying Applications to Application Clusters	161
Packages for Deploying Applications to Clusters	162
Deploying FaaS Serverless Applications	163
Application Data	164
Data Schemas and Structures	164
Cloud Native Application Storage Infrastructure	165
Application Connectivity	165
Service Discovery	166
Conclusion	168
11. Building Servers as Code.....	169
What's on a Server	170
Where Things Come From	171
Server Configuration Code	172
Server Configuration Code Modules	173
Designing Server Configuration Code Modules	174
Versioning and Promoting Server Code	175
Server Roles	175
Testing Server Code	176
Progressively Testing Server Code	177
What to Test with Server Code	177
How to Test Server Code	178
Creating a New Server Instance	179
Hand-Building a New Server Instance	180
Using a Script to Create a Server	181
Using a Stack Management Tool to Create a Server	181

Configuring the Platform to Automatically Create Servers	182
Using a Networked Provisioning Tool to Build a Server	182
Prebuilding Servers	183
Hot-Cloning a Server	184
Using a Server Snapshot	184
Creating a Clean Server Image	185
Configuring a New Server Instance	185
Frying a Server Instance	186
Baking Server Images	187
Combining Baking and Frying	188
Applying Server Configuration When Creating a Server	189
Conclusion	189
12. Managing Changes to Servers.....	191
Change Management Patterns: When to Apply Changes	192
Antipattern: Apply On Change	192
Pattern: Continuous Configuration Synchronization	194
Pattern: Immutable Server	195
How to Apply Server Configuration Code	198
Pattern: Push Server Configuration	198
Pattern: Pull Server Configuration	200
Other Server Life Cycle Events	202
Stopping and Restarting a Server Instance	202
Replacing a Server Instance	203
Recovering a Failed Server	204
Conclusion	205
13. Server Images as Code.....	207
Building a Server Image	208
Why Build a Server Image?	208
How to Build a Server Image	209
Tools for Building Server Images	209
Online Image Building Process	210
Offline Image Building Process	213
Origin Content for a Server Image	214
Building from a Stock Server Image	215
Building a Server Image from Scratch	215
Provenance of a Server Image and its Content	215
Changing a Server Image	216
Reheating or Baking a Fresh Image	216
Versioning a Server Image	217
Updating Server Instances When an Image Changes	218

Providing and Using a Server Image Across Teams	220
Handling Major Changes to an Image	220
Using a Pipeline to Test and Deliver a Server Image	221
Build Stage for a Server Image	222
Test Stage for a Server Image	223
Delivery Stages for a Server Image	224
Using Multiple Server Images	225
Server Images for Different Infrastructure Platforms	225
Server Images for Different Operating Systems	225
Server Images for Different Hardware Architectures	226
Server Images for Different Roles	226
Layering Server Images	226
Sharing Code Across Server Images	228
Conclusion	228
14. Building Clusters as Code.....	229
Application Cluster Solutions	230
Cluster as a Service	230
Packaged Cluster Distribution	231
Stack Topologies for Application Clusters	232
Monolithic Stack Using Cluster as a Service	233
Monolithic Stack for a Packaged Cluster Solution	234
Pipeline for a Monolithic Application Cluster Stack	235
Example of Multiple Stacks for a Cluster	238
Sharing Strategies for Application Clusters	241
One Big Cluster for Everything	242
Separate Clusters for Delivery Stages	242
Clusters for Governance	243
Clusters for Teams	244
Service Mesh	244
Infrastructure for FaaS Serverless	246
Conclusion	248

Part IV. Designing Infrastructure

15. Core Practice: Small, Simple Pieces.....	251
Designing for Modularity	252
Characteristics of Well-Designed Components	252
Rules for Designing Components	253
Use Testing to Drive Design Decisions	256
Modularizing Infrastructure	256

Stack Components Versus Stacks as Components	257
Using a Server in a Stack	259
Drawing Boundaries Between Components	262
Align Boundaries with Natural Change Patterns	262
Align Boundaries with Component Life Cycles	263
Align Boundaries with Organizational Structures	265
Create Boundaries That Support Resilience	265
Create Boundaries That Support Scaling	266
Align Boundaries to Security and Governance Concerns	269
Conclusion	270
16. Building Stacks from Components.	271
Infrastructure Languages for Stack Components	272
Reuse Declarative Code with Modules	272
Dynamically Create Stack Elements with Libraries	273
Patterns for Stack Components	274
Pattern: Facade Module	274
Antipattern: Obfuscation Module	276
Antipattern: Unshared Module	277
Pattern: Bundle Module	278
Antipattern: Spaghetti Module	280
Pattern: Infrastructure Domain Entity	283
Building an Abstraction Layer	284
Conclusion	285
17. Using Stacks as Components.	287
Discovering Dependencies Across Stacks	287
Pattern: Resource Matching	288
Pattern: Stack Data Lookup	291
Pattern: Integration Registry Lookup	294
Dependency Injection	296
Conclusion	299

Part V. Delivering Infrastructure

18. Organizing Infrastructure Code.	303
Organizing Projects and Repositories	303
One Repository, or Many?	304
One Repository for Everything	304
A Separate Repository for Each Project (Microrepo)	307
Multiple Repositories with Multiple Projects	308

Organizing Different Types of Code	309
Project Support Files	309
Cross-Project Tests	310
Dedicated Integration Test Projects	311
Organize Code by Domain Concept	312
Organizing Configuration Value Files	312
Managing Infrastructure and Application Code	313
Delivering Infrastructure and Applications	314
Testing Applications with Infrastructure	315
Testing Infrastructure Before Integrating	316
Using Infrastructure Code to Deploy Applications	317
Conclusion	319
19. Delivering Infrastructure Code.....	321
Delivering Infrastructure Code	321
Building an Infrastructure Project	322
Packaging Infrastructure Code as an Artifact	323
Using a Repository to Deliver Infrastructure Code	323
Integrating Projects	326
Pattern: Build-Time Project Integration	327
Pattern: Delivery-Time Project Integration	330
Pattern: Apply-Time Project Integration	333
Using Scripts to Wrap Infrastructure Tools	335
Assembling Configuration Values	336
Simplifying Wrapper Scripts	337
Conclusion	338
20. Team Workflows.....	339
The People	340
Who Writes Infrastructure Code?	342
Applying Code to Infrastructure	344
Applying Code from Your Local Workstation	344
Applying Code from a Centralized Service	345
Personal Infrastructure Instances	347
Source Code Branches in Workflows	348
Preventing Configuration Drift	349
Minimize Automation Lag	349
Avoid Ad Hoc Apply	350
Apply Code Continuously	350
Immutable Infrastructure	351
Governance in a Pipeline-based Workflow	352
Reshuffling Responsibilities	352

Shift Left	353
An Example Process for Infrastructure as Code with Governance	353
Conclusion	354
21. Safely Changing Infrastructure.....	355
Reduce the Scope of Change	355
Small Changes	358
Example of Refactoring	359
Pushing Incomplete Changes to Production	361
Parallel Instances	361
Backward Compatible Transformations	364
Feature Toggles	366
Changing Live Infrastructure	368
Infrastructure Surgery	370
Expand and Contract	372
Zero Downtime Changes	375
Continuity	376
Continuity by Preventing Errors	377
Continuity by Fast Recovery	378
Continuous Disaster Recovery	379
Chaos Engineering	379
Planning for Failure	380
Data Continuity in a Changing System	382
Lock	382
Segregate	382
Replicate	383
Reload	383
Mixing Data Continuity Approaches	384
Conclusion	384
Index.....	385

Preface

Ten years ago, a CIO at a global bank scoffed when I suggested they look into private cloud technologies and infrastructure automation tooling: “That kind of thing might be fine for startups, but we’re too large and our requirements are too complex.” Even a few years ago, many enterprises considered using public clouds to be out of the question.

These days cloud technology is pervasive. Even the largest, most hidebound organizations are rapidly adopting a “cloud-first” strategy. Those organizations that find themselves unable to consider public clouds are adopting dynamically provisioned infrastructure platforms in their data centers.¹ The capabilities that these platforms offer are evolving and improving so quickly that it’s hard to ignore them without risking obsolescence.

Cloud and automation technologies remove barriers to making changes to production systems, and this creates new challenges. While most organizations want to speed up their pace of change, they can’t afford to ignore risks and the need for governance. Traditional processes and techniques for changing infrastructure safely are not designed to cope with a rapid pace of change. These ways of working tend to throttle the benefits of modern, Cloud Age technologies—slowing work down and harming stability.²

1 For example, many government and financial organizations in countries without a cloud presence are prevented by law from hosting data or transactions abroad.

2 The research published by DORA in the *State of DevOps Report* finds that heavyweight change-management processes correlate to poor performance on change failure rates and other measures of software delivery effectiveness.

In [Chapter 1](#) I use the terms “Iron Age” and “Cloud Age” (“[From the Iron Age to the Cloud Age](#)” on [page 2](#)) to describe the different philosophies that apply to managing physical infrastructure, where mistakes are slow and costly to correct, and managing virtual infrastructure, where mistakes can be quickly detected and fixed.

Infrastructure as Code tools create the opportunity to work in ways that help you to deliver changes more frequently, quickly, and reliably, improving the overall quality of your systems. But the benefits don’t come from the tools themselves. They come from how you use them. The trick is to leverage the technology to embed quality, reliability, and compliance into the process of making changes.

Why I Wrote This Book

I wrote the first edition of this book because I didn’t see a cohesive collection of guidance on how to manage Infrastructure as Code. There was plenty of advice scattered across blog posts, conference talks, and documentation for products and projects. But a practitioner needed to sift through everything and piece a strategy together for themselves, and most people simply didn’t have time.

The experience of writing the first edition was amazing. It gave me the opportunity to travel and to talk with people around the world about their own experiences. These conversations gave me new insights and exposed me to new challenges. I learned that the value of writing a book, speaking at conferences, and consulting with clients is that it fosters conversations. As an industry, we are still gathering, sharing, and evolving our ideas for managing Infrastructure as Code.

What’s New and Different in This Edition

Things have moved along since the first edition came out in June 2016. That edition was subtitled “Managing Servers in the Cloud,” which reflected the fact that most infrastructure automation until that point had been focused on configuring servers. Since then, containers and clusters have become a much bigger deal, and the infrastructure action has moved to managing collections of infrastructure resources provisioned from cloud platforms—what I call *stacks* in this book.

As a result, this edition involves more coverage of building stacks, which is the remit of tools like CloudFormation and Terraform. The view I’ve taken is that we use stack management tools to assemble collections of infrastructure that provide application runtime environments. Those runtime environments may include servers, clusters, and serverless execution environments.

I’ve changed quite a bit based on what I’ve learned about the evolving challenges and needs of teams building infrastructure. As I’ve already touched on in this preface, I see making it safe and easy to change infrastructure as the key benefit of

Infrastructure as Code. I believe people underestimate the importance of this by thinking that infrastructure is something you build and forget.

But too many teams I meet struggle to meet the needs of their organizations; they are not able to expand and scale quickly enough, support the pace of software delivery, or provide the reliability and security expected. And when we dig into the details of their challenges, it's that they are overwhelmed by the need to update, fix, and improve their systems. So I've doubled down on this as the core theme of this book.

This edition introduces three core practices for using Infrastructure as Code to make changes safely and easily:

Define everything as code

This one is obvious from the name, and creates repeatability and consistency.

Continuously test and deliver all work in progress

Each change enhances safety. It also makes it possible to move faster and with more confidence.

Build small, simple pieces that you can change independently

These are easier and safer to change than larger pieces.

These three practices are mutually reinforcing. Code is easy to track, version, and deliver across the stages of a change management process. It's easier to continuously test smaller pieces. Continuously testing each piece on its own forces you to keep a loosely coupled design.

These practices and the details of how to apply them are familiar from the world of software development. I drew on Agile software engineering and delivery practices for the first edition of the book. For this edition, I've also drawn on rules and practices for effective design.

In the past few years, I've seen teams struggle with larger and more complicated infrastructure systems, and I've seen the benefits of applying lessons learned in software design patterns and principles, so I've included several chapters in this book on how to do this.

I've also seen that organizing and working with infrastructure code is difficult for many teams, so I've addressed various pain points. I describe how to keep codebases well organized, how to provide development and test instances for infrastructure, and how to manage the collaboration of multiple people, including those responsible for governance.

What's Next

I don't believe we've matured as an industry in how we manage infrastructure. I'm hoping this book gives a decent view of what teams are finding effective these days. And a bit of aspiration of what we can do better.

I fully expect that in another five years the toolchains and approaches will evolve. We could see more general-purpose languages used to build libraries, and we could be dynamically generating infrastructure rather than defining the static details of environments at a low level. We certainly need to get better at managing changes to live infrastructure. Most teams I know are scared when applying code to live infrastructure. (One team referred to Terraform as “Terrorform,” but users of other tools all feel this way.)

What This Book Is and Isn't

The thesis of this book is that exploring different ways of using tools to implement infrastructure can help us to improve the quality of services we provide. We aim to use speed and frequency of delivery to improve the reliability and quality of what we deliver.

So the focus of this book is less on specific tools, and more on how to use them.

Although I mention examples of tools for particular functions like configuring servers and provisioning stacks, you won't find details of how to use a particular tool or cloud platform. You will find patterns, practices, and techniques that should be relevant to whatever tools and platforms you use.

You won't find code examples for real-world tools or clouds. Tools change too quickly in this field to keep code examples accurate, but the advice in this book should age more slowly, and be applicable across tools. Instead, I write pseudocode examples for fictional tools to illustrate concepts. See the book's [companion website](#) for references to example projects and code.

This book won't guide you on how to use the Linux operating system, Kubernetes cluster configuration, or network routing. The scope of this book does include ways to provision infrastructure resources to create these things, and how to use code to deliver them. I share different cluster topology patterns and approaches for defining and managing clusters as code. I describe patterns for provisioning, configuring, and changing server instances using code.

You should supplement the practices in this book with resources on the specific operating systems, clustering technologies, and cloud platforms. Again, this book explains approaches for using these tools and technologies that are relevant regardless of the particular tool.

This book is also light on operability topics like monitoring and observability, log aggregation, identity management, and other concerns that you need to support services in a cloud environment. What’s in here should help you to manage the infrastructure needed for these services as code, but the details of the specific services are, again, something you’ll find in more specific resources.

Some History of Infrastructure as Code

Infrastructure as Code tools and practices emerged well before the term. Systems administrators have been using scripts to help them manage systems since the beginning. Mark Burgess created the pioneering **CFEngine** system in 1993. I first learned practices for using code to fully automate provisioning and updates of servers from the **Infrastructures.org** website in the early 2000s.³

Infrastructure as Code has grown along with the DevOps movement. Andrew Clay-Shafer and Patrick Debois triggered the DevOps movement with **a talk at the Agile 2008 conference**. The first uses I’ve found for the term “Infrastructure as Code” are from a talk called **“Agile Infrastructure”** that Clay-Shafer gave at the Velocity conference in 2009, and **an article** John Willis wrote summarizing the talk. Adam Jacob, who cofounded Chef, and Luke Kanies, founder of Puppet, were also using the phrase around this time.

Who This Book Is For

This book is for people who are involved in providing and using infrastructure to deliver and run software. You may have a background in systems and infrastructure, or in software development and delivery. Your role may be engineering, testing, architecture, or management. I’m assuming you have some exposure to cloud or virtualized infrastructure and tools for automating infrastructure using code.

Readers new to Infrastructure as Code should find this book a good introduction to the topic, although you will get the most out of it if you are familiar with how infrastructure cloud platforms work, and the basics of at least one infrastructure coding tool.

Those who have more experience working with these tools should find a mixture of familiar and new concepts and approaches. The content should create a common language and articulate challenges and solutions in ways that experienced practitioners and teams find useful.

³ The original content remains on this site as of summer 2020, although it hadn’t been updated since 2007.

Principles, Practices, and Patterns

I use the terms *principles*, *practices*, and *patterns* (and *antipatterns*) to describe essential concepts. Here are the ways I use each of these terms:

Principle

A principle is a rule that helps you to choose between potential solutions.

Practice

A practice is a way of implementing something. A given practice is not always the only way to do something, and may not even be the best way to do it for a particular situation. You should use principles to guide you in choosing the most appropriate practice for a given situation.

Pattern

A pattern is a potential solution to a problem. It's very similar to a practice in that different patterns may be more effective in different contexts. Each pattern is described in a format that should help you to evaluate how relevant it is for your problem.

Antipattern

An antipattern is a potential solution that you should avoid in most situations. Usually, it's either something that seems like a good idea or else it's something that you fall into doing without realizing it.

Why I Don't Use the Term "Best Practice"

Folks in our industry love to talk about "best practices." The problem with this term is that it often leads people to think there is only one solution to a problem, no matter what the context.

I prefer to describe practices and patterns, and note when they are useful and what their limitations are. I do describe some of these as being more effective or more appropriate, but I try to be open to alternatives. For practices that I believe are less effective, I hope I explain why I think this.

The ShopSpinner Examples

I use a fictional company called ShopSpinner to illustrate concepts throughout this book. ShopSpinner builds and runs online stores for its customers.

ShopSpinner runs on FCS, the Fictional Cloud Service, a public IaaS provider with services that include FSI (Fictional Server Images) and FKS (Fictional Kubernetes Service). It uses the *Stackmaker* tool—an analog of Terraform, CloudFormation, and

Pulumi—to define and manage infrastructure on its cloud. It configures servers with the *Servermaker* tool, which is much like Ansible, Chef, or Puppet.

ShopSpinner’s infrastructure and system design may vary depending on the point I’m using it to make, as will the syntax of the code and command-line arguments for its fictional tools.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

O'Reilly Online Learning



For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <http://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/infra-as-code-2e>.

Email bookquestions@oreilly.com to comment or ask technical questions about this book.

For news and information about our books and courses, visit <http://oreilly.com>.

Find us on Facebook: <http://facebook.com/oreilly>

Follow us on Twitter: <http://twitter.com/oreillymedia>

Watch us on YouTube: <http://www.youtube.com/oreillymedia>

Acknowledgments

As with the first edition, this book is not my product—it's the result of collating and consolidating as best I could what I've learned from more people than I'm able to remember and properly credit. Apologies and thanks to those I've forgotten to name here.

I always enjoy knocking ideas around with James Lewis; our conversations and his writing and talks have directly and indirectly influenced much of what's in this book. He kindly shared his deep experience on software design and broad awareness of various other topics by giving me thoughts on a near-final draft of the book. His suggestions helped me to tighten up the connections I've tried to draw between software engineering and Infrastructure as Code.

Martin Fowler has generously supported my efforts from the beginning. His skill at drawing from various people's experiences, applying his knowledge and perception, and shaping it all into clear, helpful advice inspires me.

Thierry de Pauw has been a most thoughtful and helpful reviewer. He read multiple drafts and shared his reactions, telling me what he found new and useful, which ideas aligned with his own experiences, and which parts didn't come across clearly to him.

I need to thank Abigail Bangser, Jon Barber, Max Griffiths, Anne Simmons, and Claire Walkley for their encouragement and inspiration.

People I've worked with gave me thoughts and ideas that improved this book. James Green shared insights into data engineering and machine learning in the context of infrastructure. Pat Downey explained his use of expand and contract for infrastructure. Vincenzo Fabrizio pointed out to me the value of inversion of control for infrastructure dependencies. Effy Elden is an endless fount of knowledge about the infrastructure tooling landscape. Moritz Heiber directly and indirectly influenced the contents of this book, although it would be too much to hope that he agrees with 100% of it.

At ThoughtWorks I have the opportunity to interact with and discuss Infrastructure as Code and related topics with many colleagues and clients in workshops, projects, and online forums. A few of these people include Ama Asare, Nilakhya Chatterjee, Audrey Conceicao, Patrick Dale, Dhaval Doshi, Filip Fafara, Adam Fahie, John Fennella, Mario Fernandez, Louise Franklin, Heiko Gerin, Jarrad "Barry" Goodwin, Emily Gorcenski, James Gregory, Col Harris, Prince M Jain, Andrew Jones, Aiko Klostermann, Charles Korn, Vishwas Kumar, Punit Lad, Suyu Liu, Tom Clement Oketch, Gerald Schmidt, Boss Supanat Pothivarakorn, Rodrigo Rech, Florian Sellmayr, Vladimir Sneblin, Isha Soni, Widyasari Stella, Paul Valla, Srikanth Venugopalan, Ankit Wal, Paul Yeoh, and Jiayu Yi. Also thanks to Kent Spillner—this time I remember why.

Plenty of people reviewed various drafts of this edition of the book and shared feedback, including Artashes Arabajyan, Albert Attard, Simon Bisson, Phillip Campbell, Mario Cecchi, Carlos Conde, Bamdad Dashtban, Marc Hofer, Willem van Ketwich, Barry O'Reilly, Rob Park, Robert Quinlivan, Wasin Watthanasrisong, and Rebecca Wirfs-Brock.

Deep thanks to Virginia Wilson, my editor, who has kept me going throughout the long and grueling process of making this book happen. My colleague, John Amalanathan, turned my wonky diagrams into the slick artwork you see here, with great patience and diligence.

My employer, ThoughtWorks, has been an enormous supporter. Firstly by creating the environment for me to learn from phenomenal people, secondly by fostering a culture that encourages its members to share ideas with the industry, and thirdly, by supporting me as I worked with other ThoughtWorkers and clients to explore and test new ways of working. Ashok Subramanian, Ruth Harrison, Renee Hawkins, Ken Mugrage, Rebecca Parsons, and Gayathri Rao, among others, have helped me make this more than a personal project.

Last and most, everlasting love to Ozlem and Erel, who endured my obsession with this book. Again.

PART I

Foundations

What Is Infrastructure as Code?

If you work in a team that builds and runs IT infrastructure, then cloud and infrastructure automation technology should help you deliver more value in less time, and to do it more reliably. But in practice, it drives ever-increasing size, complexity, and diversity of things to manage.

These technologies are especially relevant as organizations become digital. “Digital” is how people in business attire say that software systems are essential to what the organization does.¹ The move to digital increases the pressure on you to do more and to do it faster. You need to add and support more services. More business activities. More employees. More customers, suppliers, and other stakeholders.

Cloud and automation tools help by making it far easier to add and change infrastructure. But many teams struggle to find enough time to keep up with the infrastructure they already have. Making it easier to create even more stuff to manage is unhelpful. As one of my clients told me, “Using cloud knocked down the walls that kept our tire fire contained.”²

Many people respond to the threat of unbounded chaos by tightening their change management processes. They hope that they can prevent chaos by limiting and controlling changes. So they wrap the cloud in chains.

1 This is as opposed to what many of the same people said a few years ago, which was that software was “not part of our core business.” After following this advice and outsourcing IT, organizations realized they were being overtaken by those run by people who see better software as a way to compete, rather than as a cost to cut.

2 According to [Wikipedia](#), a *tire fire* has two forms: “Fast-burning events, leading to almost immediate loss of control, and slow-burning pyrolysis which can continue for over a decade.”

There are two problems with this. One is that it removes the benefits of using cloud technology; the other is that users *want* the benefits of cloud technology. So users bypass the people who are trying to limit the chaos. In the worst cases, people completely ignore risk management, deciding it's not relevant in the brave new world of cloud. They embrace cowboy IT, which adds different problems.³

The premise of this book is that you can exploit cloud and automation technology to make changes easily, safely, quickly, and responsibly. These benefits don't come out of the box with automation tools or cloud platforms. They depend on the way you use this technology.



DevOps and Infrastructure as Code

DevOps is a movement to reduce barriers and friction between organizational silos—development, operations, and other stakeholders involved in planning, building, and running software. Although technology is the most visible, and in some ways simplest face of DevOps, it's culture, people, and processes that have the most impact on flow and effectiveness. Technology and engineering practices like Infrastructure as Code should be used to support efforts to bridge gaps and improve collaboration.

In this chapter, I explain that modern, dynamic infrastructure requires a “Cloud Age” mindset. This mindset is fundamentally different from the traditional “Iron Age” approach we used with static pre-cloud systems. I define three core practices for implementing Infrastructure as Code: define everything as code, continuously test and deliver everything as you work, and build your system from small, loosely coupled pieces.

Also in this chapter, I describe the reasoning behind the Cloud Age approach to infrastructure. This approach discards the false dichotomy of trading speed for quality. Instead, we use speed as a way to improve quality, and we use quality to enable delivery at speed.

From the Iron Age to the Cloud Age

Cloud Age technologies make it faster to provision and change infrastructure than traditional, Iron Age technologies (Table 1-1).

³ By “cowboy IT,” I mean people building IT systems without any particular method or consideration for future consequences. Often, people who have never supported production systems take the quickest path to get things working without considering security, maintenance, performance, and other operability concerns.

Table 1-1. Technology changes in the Cloud Age

Iron Age	Cloud Age
Physical hardware	Virtualized resources
Provisioning takes weeks	Provisioning takes minutes
Manual processes	Automated processes

However, these technologies don't necessarily make it easier to manage and grow your systems. Moving a system with **technical debt** onto unbounded cloud infrastructure accelerates the chaos.

Maybe you could use well-proven, traditional governance models to control the speed and chaos that newer technologies unleash. Thorough, up-front design, rigorous change review, and strictly segregated responsibilities will impose order!

Unfortunately, these models optimize for the Iron Age, where changes are slow and expensive. They add extra work up front, hoping to reduce the time spent making changes later. This arguably makes sense when making changes later is slow and expensive. But cloud makes changes cheap and fast. You should exploit this speed to learn and improve your system continuously. Iron Age ways of working are a massive tax on learning and improvement.

Rather than using slow-moving Iron Age processes with fast-moving Cloud Age technology, adopt a new mindset. Exploit faster-paced technology to reduce risk and improve quality. Doing this requires a fundamental change of approach and new ways of thinking about change and risk (**Table 1-2**).

Table 1-2. Ways of working in the Cloud Age

Iron Age	Cloud Age
Cost of change is high	Cost of change is low
Changes represent failure (changes must be "managed," "controlled")	Changes represent learning and improvement
Reduce opportunities to fail	Maximize speed of improvement
Deliver in large batches, test at the end	Deliver small changes, test continuously
Long release cycles	Short release cycles
Monolithic architectures (fewer, larger moving parts)	Microservices architectures (more, smaller parts)
GUI-driven or physical configuration	Configuration as Code

Infrastructure as Code is a Cloud Age approach to managing systems that embraces continuous change for high reliability and quality.

Infrastructure as Code

Infrastructure as Code is an approach to infrastructure automation based on practices from software development. It emphasizes consistent, repeatable routines for provisioning and changing systems and their configuration. You make changes to code, then use automation to test and apply those changes to your systems.

Throughout this book, I explain how to use Agile engineering practices such as Test Driven Development (TDD), Continuous Integration (CI), and Continuous Delivery (CD) to make changing infrastructure fast and safe. I also describe how modern software design can create resilient, well-maintained infrastructure. These practices and design approaches reinforce each other. Well-designed infrastructure is easier to test and deliver. Automated testing and delivery drive simpler and cleaner design.

Benefits of Infrastructure as Code

To summarize, organizations adopting Infrastructure as Code to manage dynamic infrastructure hope to achieve benefits, including:

- Using IT infrastructure as an enabler for rapid delivery of value
- Reducing the effort and risk of making changes to infrastructure
- Enabling users of infrastructure to get the resources they need, when they need it
- Providing common tooling across development, operations, and other stakeholders
- Creating systems that are reliable, secure, and cost-effective
- Make governance, security, and compliance controls visible
- Improving the speed to troubleshoot and resolve failures

Use Infrastructure as Code to Optimize for Change

Given that changes are the biggest risk to a production system, continuous change is inevitable, and making changes is the only way to improve a system, it makes sense to optimize your capability to make changes both rapidly *and* reliably.⁴ Research from the *Accelerate State of DevOps Report* backs this up. Making changes frequently and reliably is correlated to organizational success.⁵

⁴ According to Gene Kim, George Spafford, and Kevin Behr in *The Visible Ops Handbook* (IT Process Institute), changes cause 80% of unplanned outages.

⁵ Reports from the *Accelerate* research are available in the annual *State of DevOps Report*, and in the book *Accelerate* by Dr. Nicole Forsgren, Jez Humble, Gene Kim (IT Revolution Press).

There are several objections I hear when I recommend a team implement automation to optimize for change. I believe these come from misunderstandings of how you can and should use automation.

Objection: “We don’t make changes often enough to justify automating them”

We want to think that we build a system, and then it’s “done.” In this view, we don’t make many changes, so automating changes is a waste of time.

In reality, very few systems stop changing, at least not before they are retired. Some people assume that their current level of change is temporary. Others create heavy-weight change request processes to discourage people from asking for changes. These people are in denial. Most teams that are supporting actively used systems handle a continuous stream of changes.

Consider these common examples of infrastructure changes:

- An essential new application feature requires you to add a new database.
- A new application feature needs you to upgrade the application server.
- Usage levels grow faster than expected. You need more servers, new clusters, and expanded network and storage capacity.
- Performance profiling shows that the current application deployment architecture is limiting performance. You need to redeploy the applications across different application servers. Doing this requires changes to the clustering and network architecture.
- There is a newly announced security vulnerability in system packages for your OS. You need to patch dozens of production servers.
- You need to update servers running a deprecated version of the OS and critical packages.
- Your web servers experience intermittent failures. You need to make a series of configuration changes to diagnose the problem. Then you need to update a module to resolve the issue.
- You find a configuration change that improves the performance of your database.

A fundamental truth of the Cloud Age is: *Stability comes from making changes.*

Unpatched systems are not stable; they are vulnerable. If you can't fix issues as soon as you discover them, your system is not stable. If you can't recover from failure quickly, your system is not stable. If the changes you do make involve considerable downtime, your system is not stable. If changes frequently fail, your system is not stable.

Objection: “We should build first and automate later”

Getting started with Infrastructure as Code is a steep curve. Setting up the tools, services, and working practices to automate infrastructure delivery is loads of work, especially if you're also adopting a new infrastructure platform. The value of this work is hard to demonstrate before you start building and deploying services with it. Even then, the value may not be apparent to people who don't work directly with the infrastructure.

Stakeholders often pressure infrastructure teams to build new cloud-hosted systems quickly, by hand, and worry about automating it later.

There are three reasons why automating afterward is a bad idea:

- Automation should enable faster delivery, even for new things. Implementing automation after most of the work has been done sacrifices many of the benefits.
- Automation makes it easier to write automated tests for what you build. And it makes it easier to quickly fix and rebuild when you find problems. Doing this as a part of the build process helps you to build better infrastructure.
- Automating an existing system is very hard. Automation is a part of a system's design and implementation. To add automation to a system built without it, you need to change the design and implementation of that system significantly. This is also true for automated testing and deployment.

Cloud infrastructure built without automation becomes a write-off sooner than you expect. The cost of manually maintaining and fixing the system can escalate quickly. If the service it runs is successful, stakeholders will pressure you to expand and add features rather than stopping to rebuild.

The same is true when you build a system as an experiment. Once you have a proof of concept up and running, there is pressure to move on to the next thing, rather than to go back and build it right. And in truth, automation should be a part of the experiment. If you intend to use automation to manage your infrastructure, you need to understand how this will work, so it should be part of your proof of concept.

The solution is to build your system incrementally, automating as you go. Ensure you deliver a steady stream of value, while also building the capability to do so continuously.

Objection: “We must choose between speed and quality”

It’s natural to think that you can only move fast by skimping on quality, and that you can only get quality by moving slowly. You might see this as a continuum, as shown in [Figure 1-1](#).

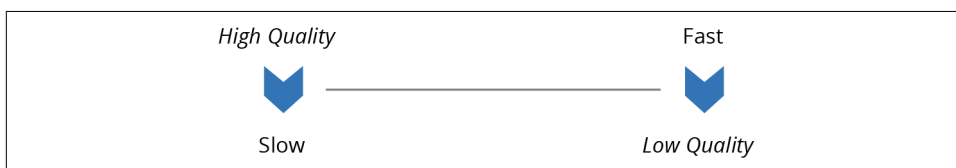


Figure 1-1. The idea that speed and quality are opposite ends of a spectrum is a false dichotomy

However, the *Accelerate* research I mentioned earlier (see “[Use Infrastructure as Code to Optimize for Change](#)” on page 4) shows otherwise:

These results demonstrate that there is no tradeoff between improving performance and achieving higher levels of stability and quality. Rather, high performers do better at all of these measures. This is precisely what the Agile and Lean movements predict, but much dogma in our industry still rests on the false assumption that moving faster means trading off against other performance goals, rather than enabling and reinforcing them.

—Nicole Forsgren, PhD, *Accelerate*

In short, organizations can’t choose between being good at change or being good at stability. They tend to either be good at both or bad at both.

I prefer to see quality and speed as a quadrant rather than a continuum,⁶ as shown in [Figure 1-2](#).

⁶ Yes, I do work at a consultancy, why do you ask?

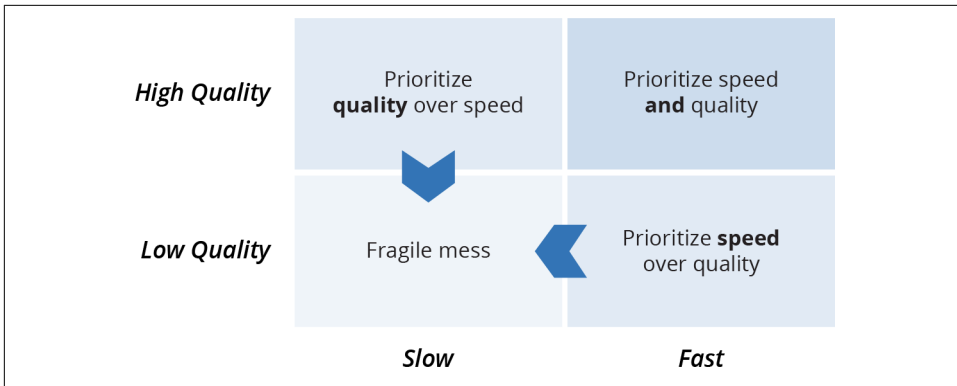


Figure 1-2. Speed and quality map to quadrants

This quadrant model shows why trying to choose between speed and quality leads to being mediocre at both:

Lower-right quadrant: Prioritize speed over quality

This is the “move fast and break things” philosophy. Teams that optimize for speed and sacrifice quality build messy, fragile systems. They slide into the lower-left quadrant because their shoddy systems slow them down. Many startups that have been working this way for a while complain about losing their “mojo.” Simple changes that they would have whipped out quickly in the old days now take days or weeks because the system is a tangled mess.

Upper-left quadrant: Prioritize quality over speed

Also known as, “We’re doing serious and important things, so we have to do things *properly*.” Then deadline pressures drive “workarounds.” Heavyweight processes create barriers to improvement, so technical debt grows along with lists of “known issues.” These teams slump into the lower-left quadrant. They end up with low-quality systems *because* it’s too hard to improve them. They add more processes in response to failures. These processes make it even harder to make improvements and increases fragility and risk. This leads to more failures and more process. Many people working in organizations that work this way assume this is normal,⁷ especially those who work in risk-sensitive industries.⁸

⁷ This is an example of “Normalization of Deviance,” which means people get used to working in ways that increase risk. Diane Vaughan defined this term in *The Challenger Launch Decision* (University Of Chicago Press).

⁸ It’s ironic (and scary) that so many people in industries like finance, government, and health care consider fragile IT systems—and processes that obstruct improving them—to be normal, and even desirable.

The upper-right quadrant is the goal of modern approaches like Lean, Agile, and DevOps. Being able to move quickly and maintain a high level of quality may seem like a fantasy. However, the *Accelerate* research proves that many teams do achieve this. So this quadrant is where you find “high performers.”

The Four Key Metrics

DORA’s *Accelerate* research team identifies four key metrics for software delivery and operational performance.⁹ Its research surveys various measures, and has found that these four have the strongest correlation to how well an organization meets its goals:

Delivery lead time

The elapsed time it takes to implement, test, and deliver changes to the production system

Deployment frequency

How often you deploy changes to production systems

Change fail percentage

What percentage of changes either cause an impaired service or need immediate correction, such as a rollback or emergency fix

Mean Time to Restore (MTTR)

How long it takes to restore service when there is an unplanned outage or impairment

Organizations that perform well against their goals—whether that’s revenue, share price, or other criteria—also perform well against these four metrics, and vice versa. The ideas in this book aim to help your team, and your organization, perform well on these metrics. Three core practices for Infrastructure as Code can help you to achieve this.

Three Core Practices for Infrastructure as Code

The Cloud Age concept exploits the dynamic nature of modern infrastructure and application platforms to make changes frequently and reliably. Infrastructure as Code is an approach to building infrastructure that embraces continuous change for high reliability and quality. So how can your team do this?

There are three core practices for implementing Infrastructure as Code:

⁹ DORA, now part of Google, is the team behind the *Accelerate State of DevOps Report*.

- Define everything as code
- Continuously test and deliver all work in progress
- Build small, simple pieces that you can change independently

I'll summarize each of these now, to set the context for further discussion. Later, I'll devote a chapter to the principles for implementing each of these practices.

Core Practice: Define Everything as Code

Defining all your stuff “as code” is a core practice for making changes rapidly and reliably. There are a few reasons why this helps:

Reusability

If you define a thing as code, you can create many instances of it. You can repair and rebuild your things quickly, and other people can build identical instances of the thing.

Consistency

Things built from code are built the same way every time. This makes system behavior predictable, makes testing more reliable, and enables continuous testing and delivery.

Transparency

Everyone can see how the thing is built by looking at the code. People can review the code and suggest improvements. They can learn things to use in other code, gain insight to use when troubleshooting, and review and audit for compliance.

I'll expand on concepts and implementation principles for defining things as code in [Chapter 4](#).

Core Practice: Continuously Test and Deliver All Work in Progress

Effective infrastructure teams are rigorous about testing. They use automation to deploy and test each component of their system, and integrate all the work everyone has in progress. They test as they work, rather than waiting until they've finished.

The idea is to *build quality in* rather than trying to *test quality in*.

One part of this that people often overlook is that it involves integrating and testing *all work in progress*. On many teams, people work on code in separate branches and only integrate when they finish. According to the *Accelerate* research, however, teams get better results when everyone integrates their work at least daily. CI involves merging and testing everyone's code throughout development. CD takes this further, keeping the merged code always production-ready.

I'll go into more detail on how to continuously test and deliver infrastructure code in [Chapter 8](#).

Core Practice: Build Small, Simple Pieces That You Can Change Independently

Teams struggle when their systems are large and tightly coupled. The larger a system is, the harder it is to change, and the easier it is to break.

When you look at the codebase of a high-performing team, you see the difference. The system is composed of small, simple pieces. Each piece is easy to understand and has clearly defined interfaces. The team can easily change each component on its own, and can deploy and test each component in isolation.

I dig more deeply into implementation principles for this core practice in [Chapter 15](#).

Conclusion

To get the value of cloud and infrastructure automation, you need a Cloud Age mindset. This means exploiting speed to improve quality, and building quality in to gain speed. Automating your infrastructure takes work, especially when you're learning how to do it. But doing it helps you to make changes, including building the system in the first place.

I've described the parts of a typical infrastructure system, as these provide the foundations for chapters explaining how to implement Infrastructure as Code.

Finally, I defined three core practices for Infrastructure as Code: defining everything as code, continuously testing and delivering, and building small pieces.

Principles of Cloud Age Infrastructure

Computing resources in the Iron Age of IT were tightly coupled to physical hardware. We assembled CPUs, memory, and hard drives in a case, mounted it into a rack, and cabled it to switches and routers. We installed and configured an operating system and application software. We could describe where an application server was in the data center: which floor, which row, which rack, which slot.

The Cloud Age decouples the computing resources from the physical hardware they run on. The hardware still exists, of course, but servers, hard drives, and routers float across it. These are no longer physical things, having transformed into virtual constructs that we create, duplicate, change, and destroy at will.

This transformation has forced us to change how we think about, design, and use computing resources. We can't rely on the physical attributes of our application server to be constant. We must be able to add and remove instances of our systems without ceremony, and we need to be able to easily maintain the consistency and quality of our systems even as we rapidly expand their scale.

There are several principles for designing and implementing infrastructure on cloud platforms. These principles articulate the reasoning for using the three core practices (define everything as code, continuously test and deliver, build small pieces). I also list several common pitfalls that teams make with dynamic infrastructure.

These principles and pitfalls underlie more specific advice on implementing Infrastructure as Code practices throughout this book.

Principle: Assume Systems Are Unreliable

In the Iron Age, we assumed our systems were running on reliable hardware. In the Cloud Age, you need to assume your system runs on unreliable hardware.¹

Cloud scale infrastructure involves hundreds of thousands of devices, if not more. At this scale, failures happen even when using reliable hardware—and most cloud vendors use cheap, less reliable hardware, detecting and replacing it when it breaks.

You'll need to take parts of your system offline for reasons other than unplanned failures. You'll need to patch and upgrade systems. You'll resize, redistribute load, and troubleshoot problems.

With static infrastructure, doing these things means taking systems offline. But in many modern organizations, taking systems offline means taking the business offline.

So you can't treat the infrastructure your system runs on as a stable foundation. Instead, you must design for uninterrupted service when underlying resources change.²

Principle: Make Everything Reproducible

One way to make a system recoverable is to make sure you can rebuild its parts effortlessly and reliably.

Effortlessly means that there is no need to make any decisions about how to build things. You should define things such as configuration settings, software versions, and dependencies as code. Rebuilding is then a simple “yes/no” decision.

Not only does reproducibility make it easy to recover a failed system, but it also helps you to:

- Make testing environments consistent with production
- Replicate systems across regions for availability
- Add instances on demand to cope with high load
- Replicate systems to give each customer a dedicated instance

1 I learned this idea from Sam Johnson's article, “[Simplifying Cloud: Reliability](#)”.

2 The principle of assuming systems are unreliable drives [chaos engineering](#), which injects failures in controlled circumstances to test and improve the reliability of your services. I talk about this more in “[Chaos Engineering](#)” on [page 379](#).

Of course, the system generates data, content, and logs, which you can't define ahead of time. You need to identify these and find ways to keep them as a part of your replication strategy. Doing this might be as simple as copying or streaming data to a backup and then restoring it when rebuilding. I'll describe options for doing this in [“Data Continuity in a Changing System” on page 382](#).

The ability to effortlessly build and rebuild any part of the infrastructure is powerful. It removes risk and fear of making changes, and you can handle failures with confidence. You can rapidly provision new services and environments.

Pitfall: Snowflake Systems

A snowflake is an instance of a system or part of a system that is difficult to rebuild. It may also be an environment that should be similar to other environments, such as a staging environment, but is different in ways that its team doesn't fully understand.

People don't set out to build snowflake systems. They are a natural occurrence. The first time you build something with a new tool, you learn lessons along the way, which involves making mistakes. But if people are relying on the thing you've built, you may not have time to go back and rebuild or improve it using what you learned. Improving what you've built is especially hard if you don't have the mechanisms and practices that make it easy and safe to change.

Another cause of snowflakes is when people make changes to one instance of a system that they don't make to others. They may be under pressure to fix a problem that only appears in one system, or they may start a major upgrade in a test environment, but run out of time to roll it out to others.

You know a system is a snowflake when you're not confident you can safely change or upgrade it. Worse, if the system does break, it's hard to fix it. So people avoid making changes to the system, leaving it out of date, unpatched, and maybe even partly broken.

Snowflake systems create risk and waste the time of the teams that manage them. It is almost always worth the effort to replace them with reproducible systems. If a snowflake system isn't worth improving, then it may not be worth keeping at all.

The best way to replace a snowflake system is to write code that can replicate the system, running the new system in parallel until it's ready. Use automated tests and pipelines to prove that it is correct and reproducible, and that you can change it easily.